

S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO



INSTITUTO TECNOLÓGICO de Tuxtepec

PROGRAMACIÓN CLIENTE - SERVIDOR

Reporte Final

PRESENTA:

Jonathan Lara Diaz

DOCENTE:

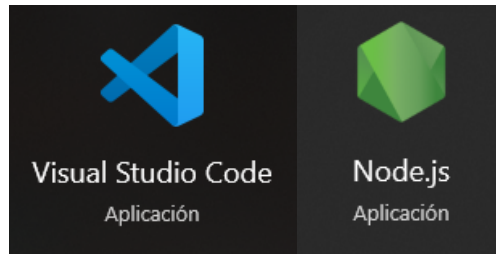
Julio Aguilar Carmona

CARRERA:

INGENIERÍA INFORMÁTICA

Dic-2025

1. Para realizar esta práctica de una pokeapi se necesitaron distintos programas, pero el principal es Visual Studio Code, Node.js y MySQL como gestor de base de datos y una estructura clara basada en rutas, controladores y modelos.
2. Se realizó la instalación de ambos programas y se realizaron las configuraciones necesarias desde la consola de Visual Studio para que los comandos que utilizemos puedan funcionar en la consola de visual o de lo contrario se tendrá que correr desde el CMD.



3. El ejecutable de la APPI

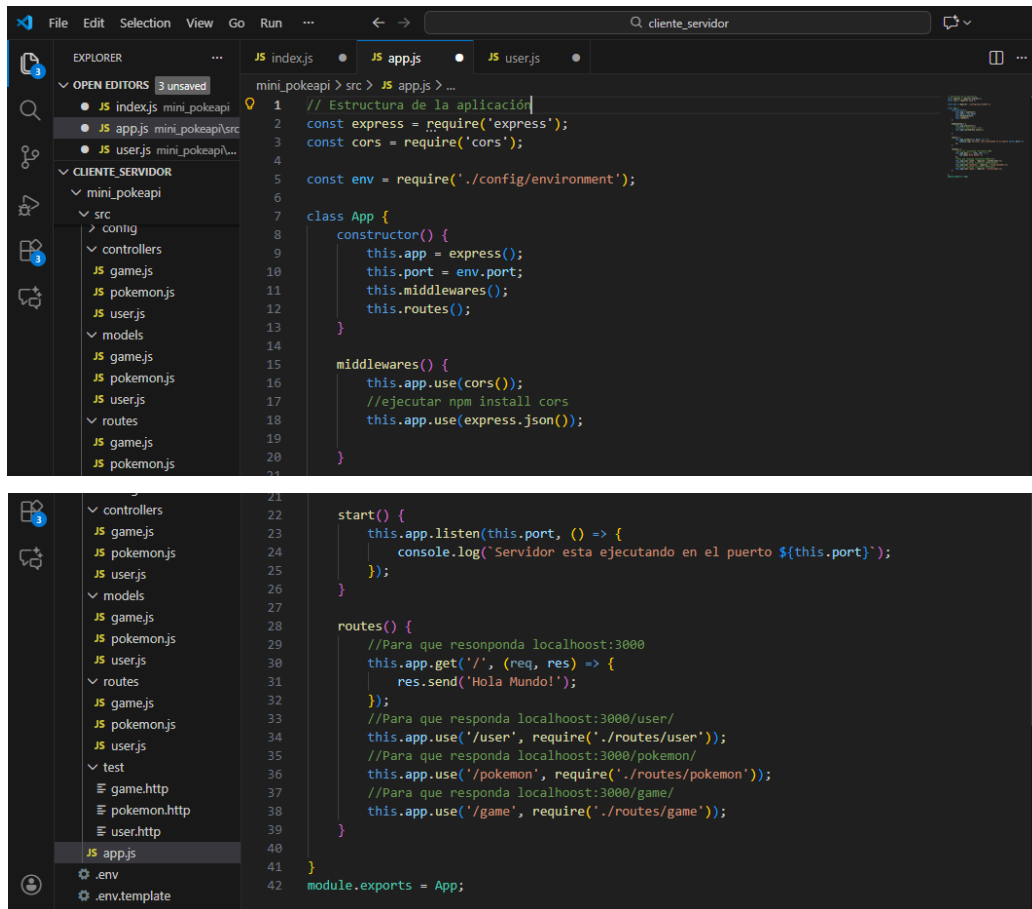
A screenshot of a code editor window. The title bar shows 'cliente_servidor'. There are three tabs: 'JS index.js mini_pokeapi', 'JS user.js', and 'JS index.js ...\js'. The active tab is 'JS index.js mini_pokeapi'. The code in the editor is as follows:

```
mini_pokeapi > JS index.js > ...
1 //EJECUTABLE DE LA APPI
2 const App = require('./src/app');
3 const app = new App();
4
5 app.start();
```

El programa es el punto de inicio de una API en Node.js. Su función principal es arrancar la aplicación.

- Primero, importa (require) la clase App desde el archivo ./src/app, donde está definida la lógica principal de la aplicación.
- Después, crea una nueva instancia de esa clase (new App()), lo que prepara la configuración y los componentes necesarios de la API.
- Finalmente, llama al método start(), que se encarga de iniciar el servidor (normalmente levantar la API, escuchar un puerto y dejarla lista para recibir solicitudes).

4. Estructura de la aplicación

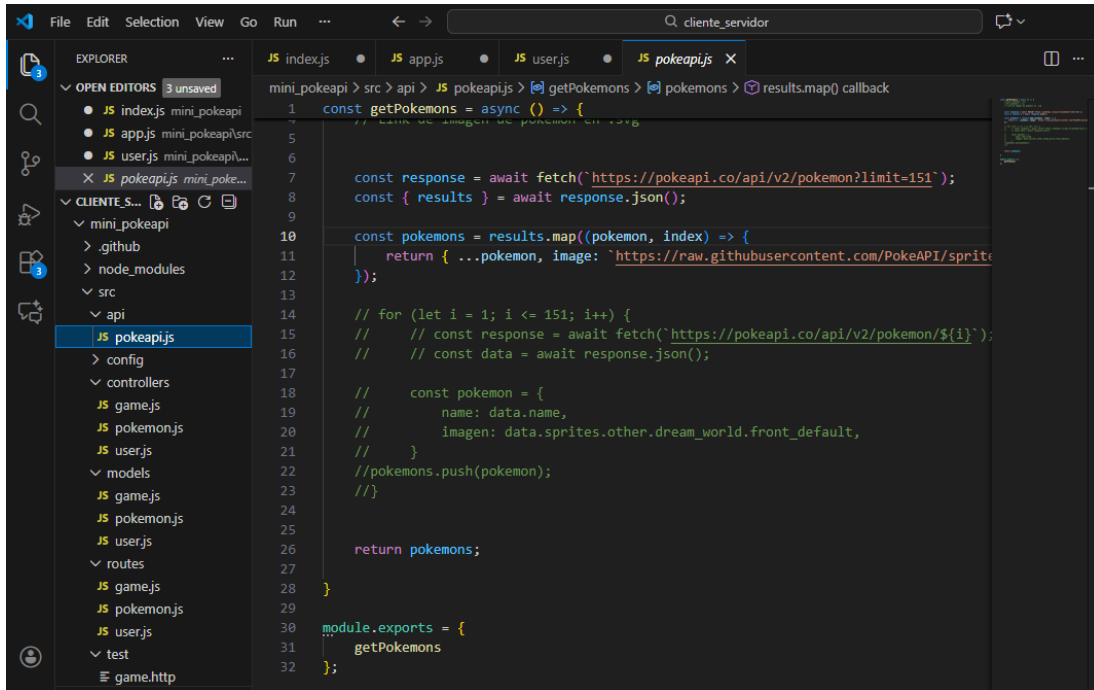


```
1 // Estructura de la aplicación
2 const express = require('express');
3 const cors = require('cors');
4
5 const env = require('./config/environment');
6
7 class App {
8   constructor() {
9     this.app = express();
10    this.port = env.port;
11    this.middlewares();
12    this.routes();
13  }
14
15   middlewares() {
16     this.app.use(cors());
17     //ejecutar npm install cors
18     this.app.use(express.json());
19   }
20
21   routes() {
22     //Para que responda localhost:3000
23     this.app.get('/', (req, res) => {
24       console.log('Servidor esta ejecutando en el puerto ${this.port}');
25     });
26
27     //Para que responda localhost:3000/user/
28     this.app.use('/user', require('./routes/user'));
29     //Para que responda localhost:3000/pokemon/
30     this.app.use('/pokemon', require('./routes/pokemon'));
31     //Para que responda localhost:3000/game/
32     this.app.use('/game', require('./routes/game'));
33   }
34
35   start() {
36     this.app.listen(this.port, () => {
37       console.log('Servidor esta ejecutando en el puerto ${this.port}');
38     });
39   }
40 }
41
42 module.exports = App;
```

Este archivo define la estructura principal de una API usando Express en Node.js. La clase App se encarga de configurar y poner en marcha el servidor.

- Primero, importa las librerías necesarias como Express para crear la API y CORS para permitir solicitudes desde otros orígenes, además de cargar la configuración del entorno (como el puerto).
- En el constructor se crea la aplicación Express, se define el puerto y se configuran los middlewares y las rutas.

5. Consulta a la PokeApi

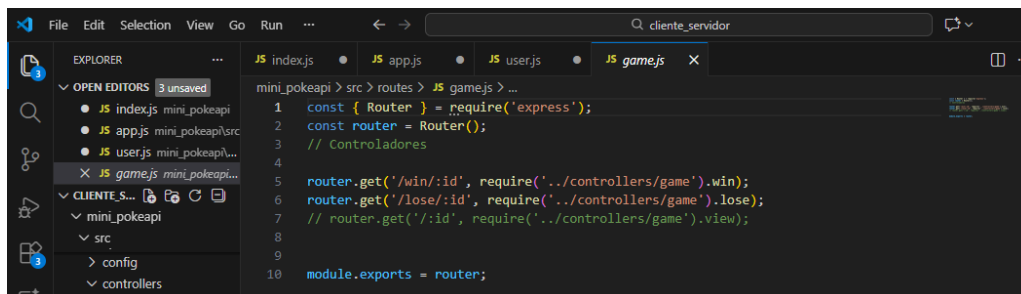


```
1 const getPokemons = async () => {  
2   // Path de imagen de pokemon en svg  
3  
4   const response = await fetch('https://pokeapi.co/api/v2/pokemon?limit=151');  
5   const { results } = await response.json();  
6  
7   const pokemons = results.map((pokemon, index) => {  
8     return { ...pokemon, image: 'https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/other/dream_world/151.png' };  
9   });  
10  
11   // for (let i = 1; i <= 151; i++) {  
12   //   // const response = await fetch(`https://pokeapi.co/api/v2/pokemon/${i}`);  
13   //   // const data = await response.json();  
14  
15   //   const pokemon = {  
16     //     name: data.name,  
17     //     imagen: data.sprites.other.dream_world.front_default,  
18   //   };  
19   //   pokemons.push(pokemon);  
20   // }  
21  
22   return pokemons;  
23 }  
24  
25 module.exports = {  
26   getPokemons  
27 };
```

Este archivo define una función asíncrona llamada `getPokemons` que obtiene información de los primeros 151 Pokémon desde la PokeAPI.

- La función realiza una petición HTTP a la API pública de Pokémon para recibir una lista básica de Pokémon. Luego, transforma esa lista usando `map`, agregando a cada Pokémon una imagen en formato SVG, construyendo el enlace de la imagen con su número correspondiente.
- Finalmente, la función devuelve un arreglo con los Pokémon, cada uno con su nombre, su información original y la URL de su imagen.

6. Controladores

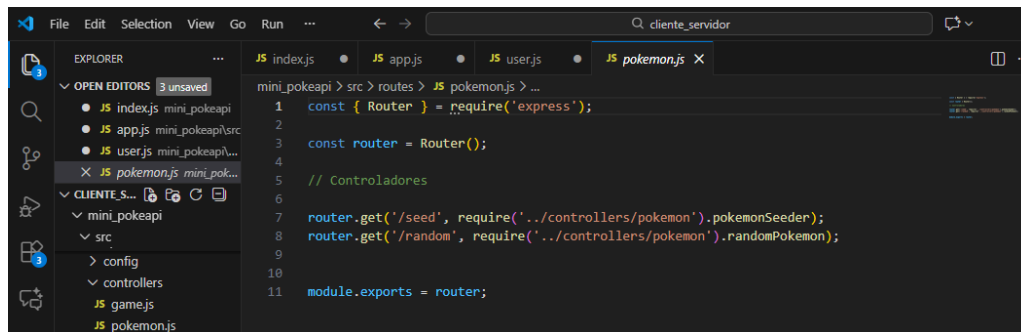


```
1 const { Router } = require('express');  
2 const router = Router();  
3 // Controladores  
4  
5 router.get('/win/:id', require('../controllers/game').win);  
6 router.get('/lose/:id', require('../controllers/game').lose);  
7 // router.get('/:id', require('../controllers/game').view);  
8  
9  
10 module.exports = router;
```

Este archivo define las rutas relacionadas con el juego dentro de una API creada con Express. Utiliza el enrutador para organizar los endpoints de forma modular.

Se crean dos rutas principales:

- `/win/:id`: se ejecuta cuando el jugador gana, recibiendo un identificador (id) como parámetro.
- `/lose/:id`: se ejecuta cuando el jugador pierde, también usando un id.

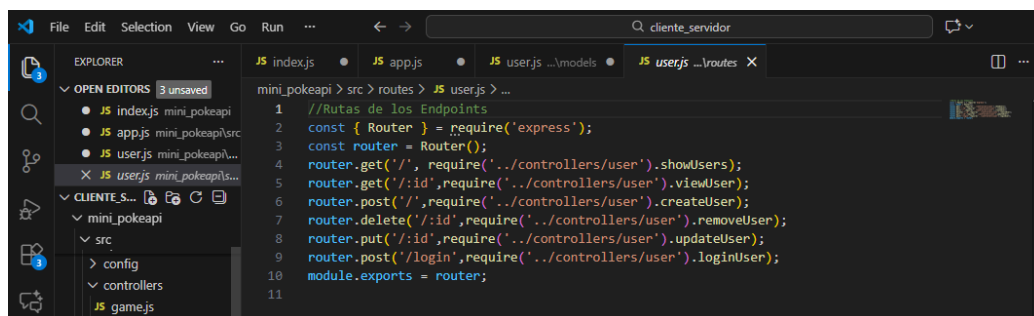


```
1 const { Router } = require('express');
2
3 const router = Router();
4
5 // Controladores
6
7 router.get('/seed', require('../controllers/pokemon').pokemonSeeder);
8 router.get('/random', require('../controllers/pokemon').randomPokemon);
9
10
11 module.exports = router;
```

Se definen dos rutas principales:

- `/seed`: ejecuta una función que carga o inicializa datos de Pokémon en el sistema.
- `/random`: devuelve un Pokémon aleatorio.

Ambas rutas delegan la lógica al controlador de Pokémon, manteniendo separada la lógica del enrutamiento.



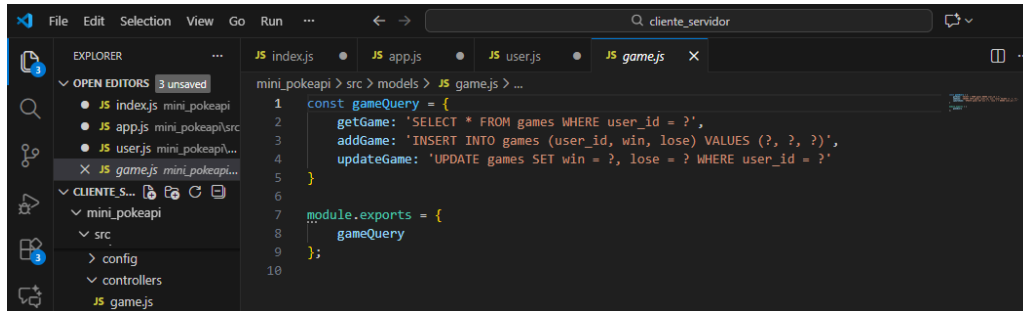
```
1 //Rutas de los Endpoints
2 const { Router } = require('express');
3 const router = Router();
4 router.get('/', require('../controllers/user').showUsers);
5 router.get('/:id', require('../controllers/user').viewUser);
6 router.post('/', require('../controllers/user').createUser);
7 router.delete('/:id', require('../controllers/user').removeUser);
8 router.put('/:id', require('../controllers/user').updateUser);
9 router.post('/login', require('../controllers/user').loginUser);
10
11 module.exports = router;
```

Este archivo define las rutas de usuarios de una API creada con Express. Utiliza el Router para organizar los endpoints de manera modular.

Permite realizar operaciones básicas sobre usuarios:

- GET `/` → muestra todos los usuarios.
- GET `/:id` → muestra un usuario específico.
- POST `/` → crea un nuevo usuario.
- PUT `/:id` → actualiza un usuario existente.
- DELETE `/:id` → elimina un usuario.

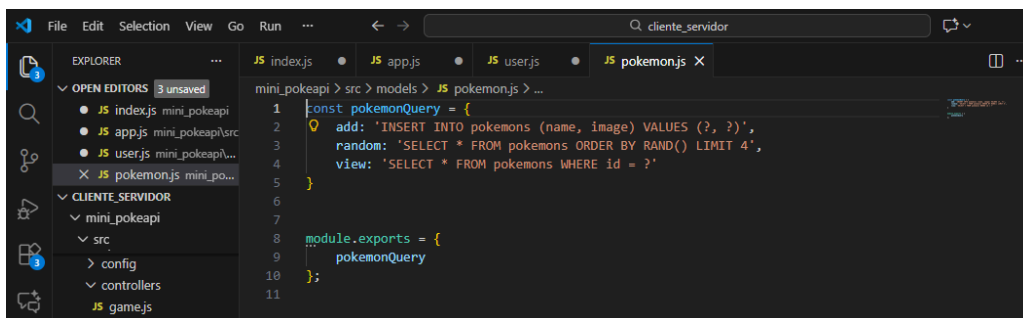
7. Interacción con la Base de Datos



```
1 const gameQuery = {
2   getGame: 'SELECT * FROM games WHERE user_id = ?',
3   addGame: 'INSERT INTO games (user_id, win, lose) VALUES (?, ?, ?)',
4   updateGame: 'UPDATE games SET win = ?, lose = ? WHERE user_id = ?'
5 }
6
7 module.exports = {
8   gameQuery
9 };
10
```

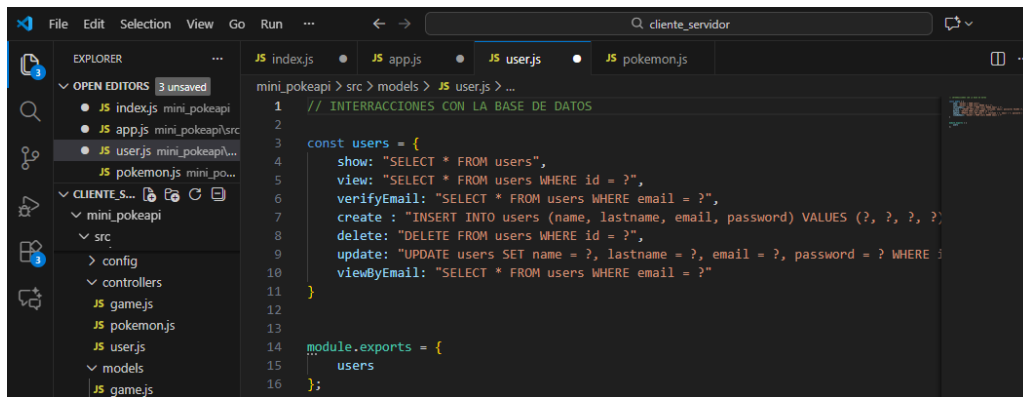
El programa define un objeto constante llamado `gameQuery` que contiene tres consultas SQL fundamentales para el manejo de datos de usuarios:

- `getGame`: Se utiliza para consultar y obtener toda la información de la tabla `games` que pertenezca a un usuario específico mediante su `user_id`.
- `addGame`: Permite registrar un nuevo usuario en la base de datos, insertando su ID junto con el número de partidas ganadas y perdidas.
- `updateGame`: Sirve para modificar los datos ya existentes, permitiendo actualizar el contador de victorias y derrotas de un jugador en particular.



```
1 const pokemonQuery = {
2   add: 'INSERT INTO pokemons (name, image) VALUES (?, ?)',
3   random: 'SELECT * FROM pokemons ORDER BY RAND() LIMIT 4',
4   view: 'SELECT * FROM pokemons WHERE id = ?'
5 }
6
7
8 module.exports = {
9   pokemonQuery
10 };
11
```

- `add`: Se utiliza para registrar un nuevo Pokémon en la base de datos, guardando su nombre y la URL o ruta de su imagen.
- `random`: Esta consulta selecciona 4 registros al azar de la tabla. Es ideal para mecánicas de juego donde el usuario debe elegir entre varias opciones o enfrentar enemigos aleatorios.
- `view`: Permite obtener toda la información de un Pokémon específico buscando directamente por su identificador único.

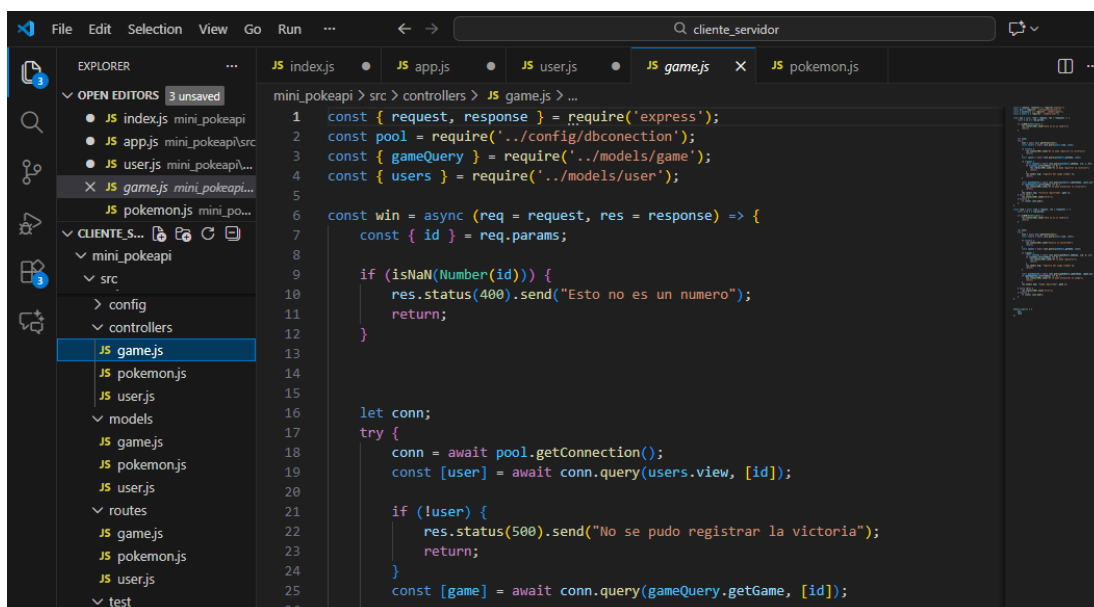


```
1 // INTERACCIONES CON LA BASE DE DATOS
2
3 const users = {
4   show: "SELECT * FROM users",
5   view: "SELECT * FROM users WHERE id = ?",
6   verifyEmail: "SELECT * FROM users WHERE email = ?",
7   create: "INSERT INTO users (name, lastname, email, password) VALUES (?, ?, ?, ?)",
8   delete: "DELETE FROM users WHERE id = ?",
9   update: "UPDATE users SET name = ?, lastname = ?, email = ?, password = ? WHERE id = ?",
10  viewByEmail: "SELECT * FROM users WHERE email = ?"
11 }
12
13 module.exports = {
14   users
15 };
```

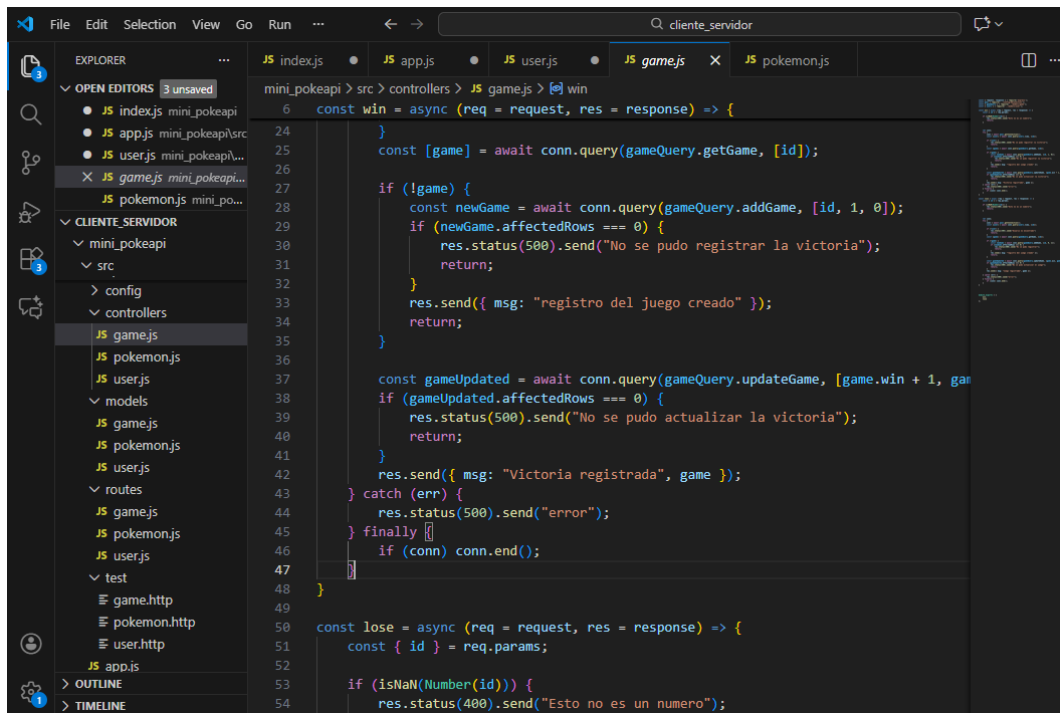
En el objeto users se definen las operaciones fundamentales para administrar la información de las personas registradas:

- show: Recupera la lista completa de todos los usuarios registrados.
- view: Busca y muestra la información detallada de un usuario utilizando su id único.
- create: Registra un nuevo usuario solicitando cuatro datos obligatorios: nombre, apellido, correo electrónico y contraseña.
- delete: Elimina permanentemente el registro de un usuario basado en su id.
- update: Permite modificar todos los datos de un usuario existente (nombre, apellido, email y contraseña) identificándolo por su id.
- verifyEmail y viewByEmail: Ambas consultas permiten buscar a un usuario a través de su correo electrónico.

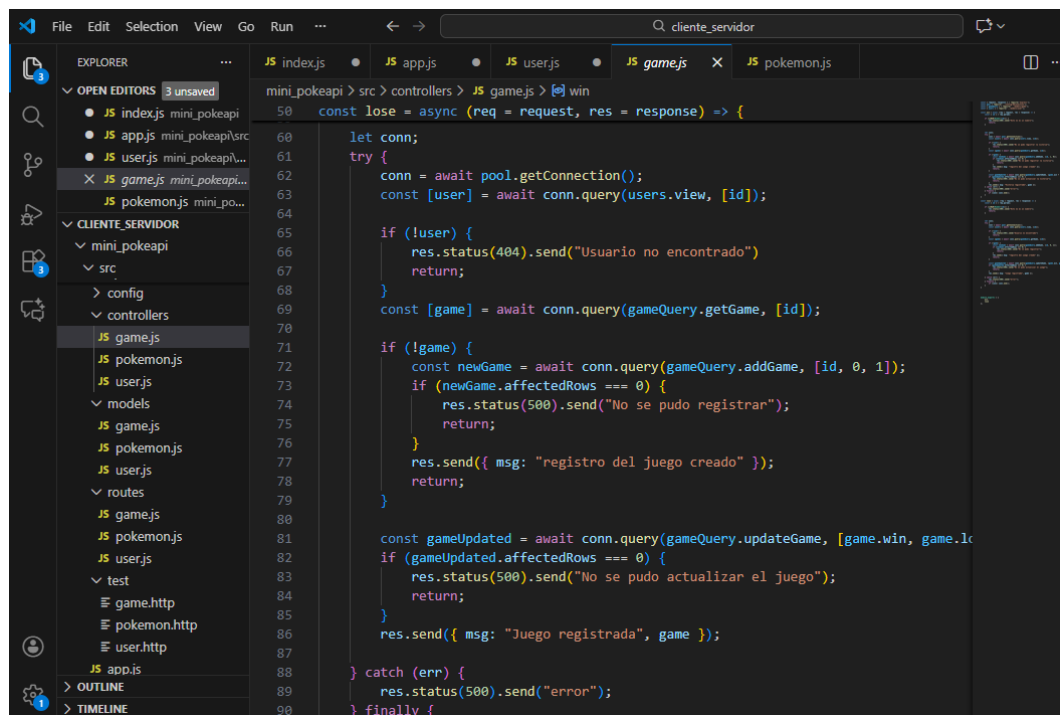
8. Acciones de los EndPoints



```
1 const { request, response } = require('express');
2 const pool = require('../config/dbconnection');
3 const { gameQuery } = require('../models/game');
4 const { users } = require('../models/user');
5
6 const win = async (req = request, res = response) => {
7   const { id } = req.params;
8
9   if (isNaN(Number(id))) {
10     res.status(400).send("Esto no es un numero");
11     return;
12   }
13
14   let conn;
15   try {
16     conn = await pool.getConnection();
17     const [user] = await conn.query(users.view, [id]);
18
19     if (!user) {
20       res.status(500).send("No se pudo registrar la victoria");
21       return;
22     }
23
24     const [game] = await conn.query(gameQuery.getGame, [id]);
25   } catch (error) {
26     console.log(error);
27   }
```



```
mini_pokeapi > src > controllers > JS gamejs > win
6  const win = async (req = request, res = response) => {
24  }
25  const [game] = await conn.query(gameQuery.getGame, [id]);
26
27  if (!game) {
28    const newGame = await conn.query(gameQuery.addGame, [id, 1, 0]);
29    if (newGame.affectedRows === 0) {
30      res.status(500).send("No se pudo registrar la victoria");
31      return;
32    }
33    res.send({ msg: "registro del juego creado" });
34    return;
35  }
36
37  const gameUpdated = await conn.query(gameQuery.updateGame, [game.win + 1, game.id]);
38  if (gameUpdated.affectedRows === 0) {
39    res.status(500).send("No se pudo actualizar la victoria");
40    return;
41  }
42  res.send({ msg: "Victoria registrada", game });
43 } catch (err) {
44   res.status(500).send("error");
45 } finally {
46   if (conn) conn.end();
47 }
48
49
50 const lose = async (req = request, res = response) => {
51   const { id } = req.params;
52
53   if (isNaN(Number(id))) {
54     res.status(400).send("Esto no es un numero");
```



```
mini_pokeapi > src > controllers > JS gamejs > view
50  const lose = async (req = request, res = response) => {
60
61  let conn;
62  try {
63    conn = await pool.getConnection();
64    const [user] = await conn.query(users.view, [id]);
65
66    if (!user) {
67      res.status(404).send("Usuario no encontrado");
68      return;
69    }
70    const [game] = await conn.query(gameQuery.getGame, [id]);
71
72    if (!game) {
73      const newGame = await conn.query(gameQuery.addGame, [id, 0, 1]);
74      if (newGame.affectedRows === 0) {
75        res.status(500).send("No se pudo registrar");
76        return;
77      }
78      res.send({ msg: "registro del juego creado" });
79      return;
80    }
81
82    const gameUpdated = await conn.query(gameQuery.updateGame, [game.win, game.loss + 1, game.id]);
83    if (gameUpdated.affectedRows === 0) {
84      res.status(500).send("No se pudo actualizar el juego");
85      return;
86    }
87    res.send({ msg: "Juego registrada", game });
88  } catch (err) {
89    res.status(500).send("error");
90  } finally {
```

Este archivo es un controlador de Node.js que contiene la lógica de negocio para gestionar el historial de victorias y derrotas de los usuarios en el juego. A diferencia de los archivos anteriores que solo contenían consultas, este programa ejecuta acciones basadas en los resultados de esas consultas.

This screenshot shows the implementation of the 'agregar' endpoint in the `pokemon.js` controller. The code uses an asynchronous function `pokemonSeeder` to handle the request. It establishes a database connection, sets foreign key checks, truncates the `pokemons` table, and then inserts the new pokemon data from the request body. A success message is sent to the client, and the connection is closed.

```
mini_pokeapi > src > controllers > JS pokemonjs > ...
1  const { request, response } = require('express');
2  const pokeapi = require('../api/pokeapi');
3  const pool = require('../config/dbconnection');
4  const { pokemonQuery } = require('../models/pokemon');
5
6  const pokemonSeeder = async (req = request, res = response) => {
7    const pokemons = await pokeapi.getPokemons();
8
9    let conn;
10
11    try {
12      conn = await pool.getConnection();
13      await conn.query('SET FOREIGN_KEY_CHECKS = 0');
14      await conn.query('TRUNCATE TABLE pokemons');
15      await conn.query('SET FOREIGN_KEY_CHECKS = 1');
16
17      pokemons.forEach(async (pokemon) => {
18        await conn.query(pokemonQuery.add, [pokemon.name, pokemon.image]);
19      });
20      res.send("Pokemones agregados en la Base de Datos");
21    } catch (err) {
22      return res.status(500).send(err);
23    } finally {
24      if (conn) {
25        conn.end();
26      }
27    }
28  }
29
30  }
```

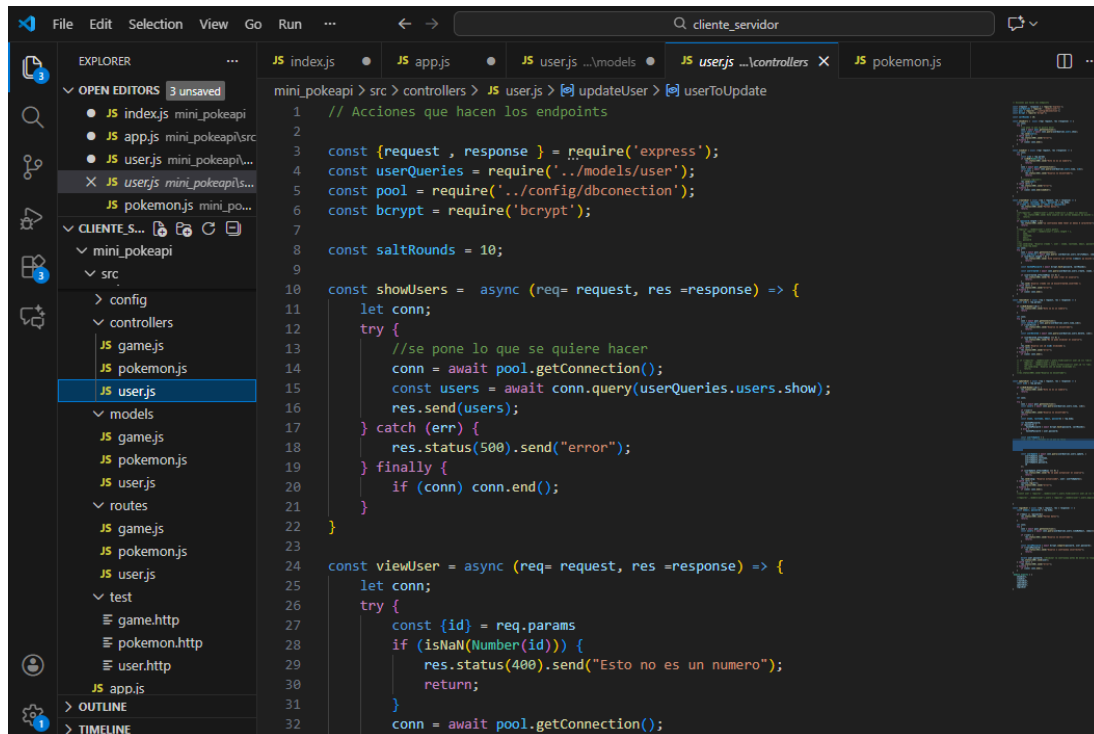
This screenshot shows the implementation of two more endpoints in the `pokemon.js` controller: `randomPokemon` and `idPokemon`. `randomPokemon` fetches a random pokemon from the database and returns its details. `idPokemon` checks if the provided ID is a valid number; if not, it returns a 400 status. If valid, it proceeds to the next step (shown in the next screenshot).

```
mini_pokeapi > src > controllers > JS pokemonjs > ...
31  const randomPokemon = async (req = request, res = response) => {
32    let conn;
33    try {
34      conn = await pool.getConnection();
35      const pokemons = await conn.query(pokemonQuery.random);
36      if (pokemons.length === 0) {
37        return res.status(500).send("No hay pokemones en la base de datos");
38      }
39      res.send(pokemons);
40    } catch (err) {
41      return res.status(500).send(err);
42    } finally {
43      if (conn) {
44        conn.end();
45      }
46    }
47  }
48
49  const idPokemon = async (req = request, res = response) => {
50    let conn;
51    try {
52      const { id } = req.params
53      if (isNaN(Number(id))) {
54        res.status(400).send("Esto no es un numero");
55        return;
56      }
57    }
58  }
```

This screenshot shows the completion of the `idPokemon` endpoint and the final module exports. The `idPokemon` function now fetches the specific pokemon by ID and returns its details, or a 404 status if not found. The `module.exports` object is defined to export the `pokemonSeeder`, `randomPokemon`, and `idPokemon` functions.

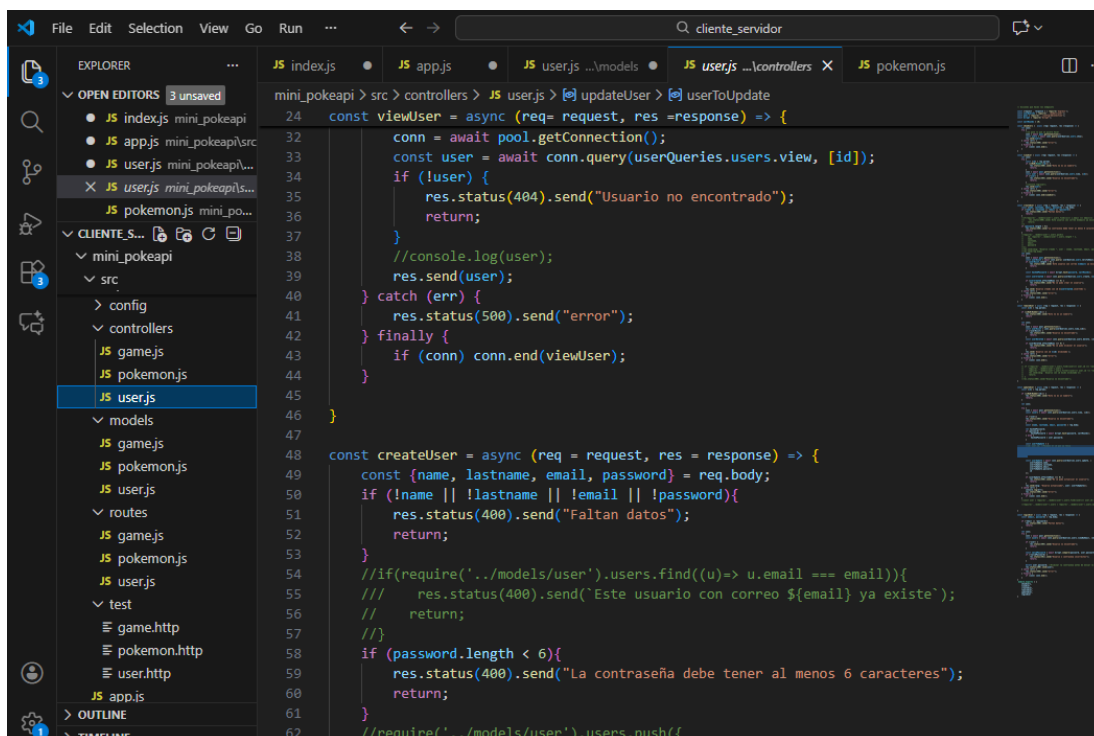
```
59
60  conn = await pool.getConnection();
61  const pokeid = await conn.query(pokemonQuery.view, [id]);
62  if (!pokeid) {
63    res.status(404).send("Pokemon no encontrado");
64    return;
65  }
66  res.send(pokeid);
67 } catch (err) {
68   return res.status(500).send(err);
69 } finally {
70   if (conn) {
71     conn.end();
72   }
73 }
74
75 module.exports = {
76   pokemonSeeder,
77   randomPokemon,
78   idPokemon
79 };
80
```

Este archivo es un controlador de Node.js encargado de gestionar la lógica de los datos de Pokémon en la aplicación. Su función principal es servir como puente entre una API externa (PokeAPI), la base de datos local y las peticiones del usuario.



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor in the center. The file explorer shows the project structure with the following folders: mini_pokeapi, CLIENTE_S..., mini_pokeapi, src, config, controllers, gamejs, pokemonjs, userjs, models, routes, gamejs, pokemonjs, userjs, test, game.http, pokemon.http, user.http, app.js. The code editor shows the 'updateUser' function in the 'userjs' controller. The code is as follows:

```
1 // Acciones que hacen los endpoints
2
3 const {request, response} = require('express');
4 const userQueries = require('../models/user');
5 const pool = require('../config/dbconnection');
6 const bcrypt = require('bcrypt');
7
8 const saltRounds = 10;
9
10 const showUsers = async (req= request, res =response) => {
11   let conn;
12   try {
13     //se pone lo que se quiere hacer
14     conn = await pool.getConnection();
15     const users = await conn.query(userQueries.users.show);
16     res.send(users);
17   } catch (err) {
18     res.status(500).send("error");
19   } finally {
20     if (conn) conn.end();
21   }
22 }
23
24 const viewUser = async (req= request, res =response) => {
25   let conn;
26   try {
27     const {id} = req.params
28     if (isNaN(Number(id))) {
29       res.status(400).send("Esto no es un numero");
30       return;
31     }
32     conn = await pool.getConnection();
```



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor in the center. The file explorer shows the project structure with the following folders: mini_pokeapi, CLIENTE_S..., mini_pokeapi, src, config, controllers, gamejs, pokemonjs, userjs, models, routes, gamejs, pokemonjs, userjs, test, game.http, pokemon.http, user.http, app.js. The code editor shows the 'viewUser' and 'createUser' functions in the 'userjs' controller. The code is as follows:

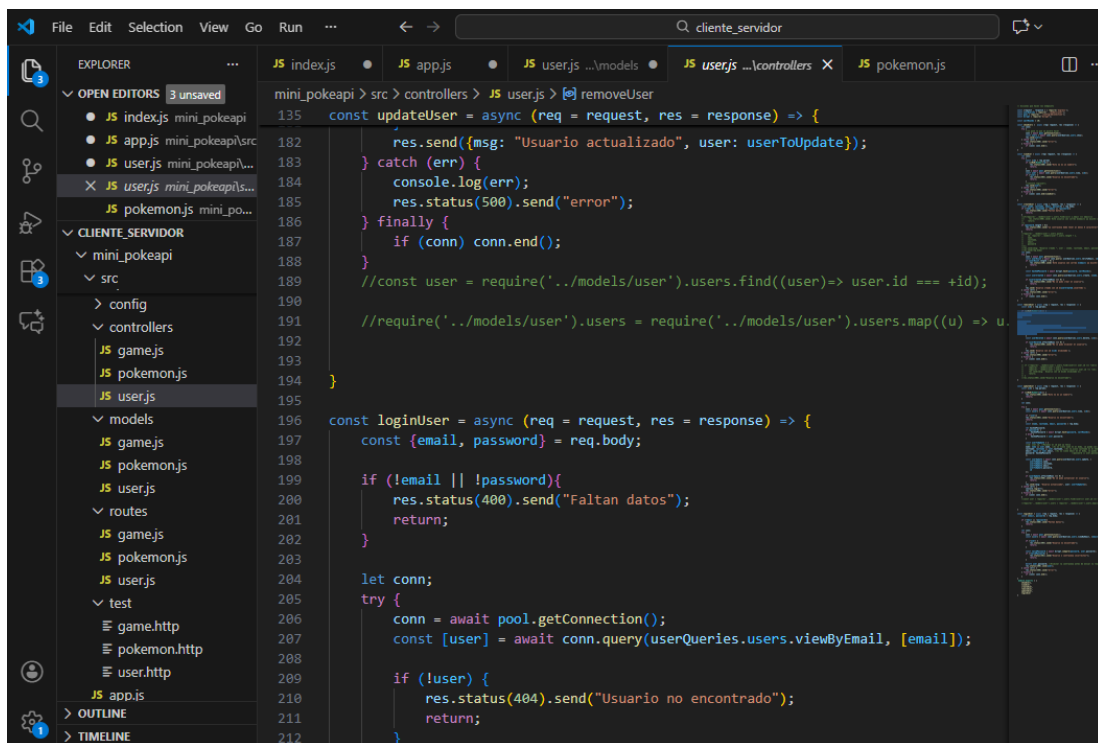
```
24 const viewUser = async (req= request, res =response) => {
25   conn = await pool.getConnection();
26   const user = await conn.query(userQueries.users.view, [id]);
27   if (!user) {
28     res.status(404).send("Usuario no encontrado");
29     return;
30   }
31   //console.log(user);
32   res.send(user);
33 } catch (err) {
34   res.status(500).send("error");
35 } finally {
36   if (conn) conn.end(viewUser);
37 }
38
39 const createUser = async (req = request, res = response) => {
40   const {name, lastname, email, password} = req.body;
41   if (!name || !lastname || !email || !password){
42     res.status(400).send("Faltan datos");
43     return;
44   }
45   //if(require('../models/user').users.find((u)=> u.email === email)){
46   //  res.status(400).send("Este usuario con correo ${email} ya existe");
47   //  return;
48   //}
49   if (password.length < 6){
50     res.status(400).send("La contraseña debe tener al menos 6 caracteres");
51     return;
52   }
53   //require('../models/user').users.push({
```

This screenshot shows the VS Code editor with the file explorer on the left, displaying a project structure for 'mini_pokeapi'. The main editor window shows the 'updateUser' endpoint in the 'userjs' controller. The code defines an asynchronous function 'createUser' that handles user creation. It includes logic to check if a user with the same email already exists and to hash the password using 'bcrypt'. The function uses a database connection pool to execute queries. The code is as follows:

```
mini_pokeapi > src > controllers > JS userjs > updateUser > userToUpdate
48 const createUser = async (req = request, res = response) => {
49   //require('../models/user').users.push({
62   // id: require('../models/user').users.length + 1,
63   // name,
64   // lastname,
65   // email,
66   // password
67   //});
68   //res.send({msg: "Usuario creado ", user : {name, lastname, email, password}});
69   //res.send(req.body)
70   let conn;
71   try {
72     conn = await pool.getConnection();
73     const userExist = await conn.query(userQueries.users.verifyEmail, [email]);
74     if (userExist.length > 0) {
75       res.status(400).send('Este usuario con correo ${email} ya existe');
76       return;
77     }
78     const hashedPassword = await bcrypt.hash(password, saltRounds);
79     const userCreated = await conn.query(userQueries.users.create, [name, lastname, hashedPassword]);
80     if (userCreated.affectedRows === 0) {
81       res.status(500).send("No se pudo crear el usuario");
82       return;
83     }
84     res.send('Usuario creado con id ${userCreated.insertId}');
85   } catch (err) {
86     res.status(500).send("error");
87   } finally {
88     if (conn) conn.end();
89   }
90 }
91
92
```

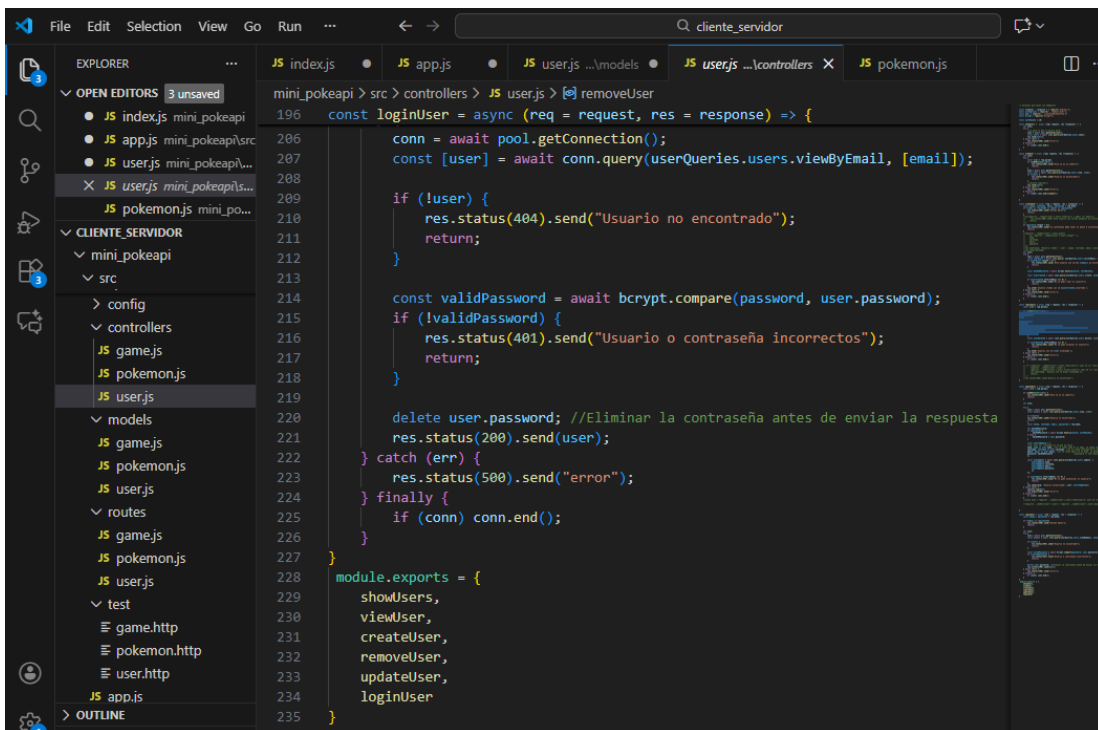
This screenshot shows the VS Code editor with the file explorer on the left, displaying a project structure for 'mini_pokeapi'. The main editor window shows the 'removeUser' endpoint in the 'userjs' controller. The code defines an asynchronous function 'removeUser' that handles user removal. It includes logic to check if the provided ID is a valid number and if the user exists. The function uses a database connection pool to execute queries. The code is as follows:

```
mini_pokeapi > src > controllers > JS userjs > updateUser > userToUpdate
48 const createUser = async (req = request, res = response) => {
87   }
88   res.send('Usuario creado con id ${userCreated.insertId}');
89 } catch (err) {
90   res.status(500).send("error");
91 } finally {
92   if (conn) conn.end();
93 }
94 }
95
96 const removeUser = async (req = request, res = response) => {
97   const {id} = req.params;
98   if (isNaN(Number(id))) {
99     res.status(400).send("Esto no es un numero");
100     return
101   }
102   let conn;
103   try {
104     conn = await pool.getConnection();
105     const userExist = conn.query(userQueries.users.view, [id]);
106     if (!userExist) {
107       res.status(404).send("Usuario no encontrado");
108       return;
109     }
110     const userDeleted = await conn.query(userQueries.users.delete, [id]);
111     if (userDeleted.affectedRows === 0) {
112       res.status(500).send("No se pudo eliminar el usuario");
113       return;
114     }
115   }
116 }
117
```

The screenshot shows the VS Code editor with the Explorer sidebar on the left. The Explorer sidebar shows the project structure: mini_pokeapi > src > controllers > JS userjs > removeUser. The main editor displays the code for the 'removeUser' function. The code is as follows:

```
135 const updateUser = async (req = request, res = response) => {
136   res.send({msg: "Usuario actualizado", user: userToUpdate});
137 } catch (err) {
138   console.log(err);
139   res.status(500).send("error");
140 } finally {
141   if (conn) conn.end();
142 }
143 //const user = require('../models/user').users.find((user)=> user.id === +id);
144 //require('../models/user').users = require('../models/user').users.map((u) => u
145 }
146
147 const loginUser = async (req = request, res = response) => {
148   const {email, password} = req.body;
149
150   if (!email || !password){
151     res.status(400).send("Faltan datos");
152     return;
153   }
154
155   let conn;
156   try {
157     conn = await pool.getConnection();
158     const [user] = await conn.query(userQueries.users.viewByEmail, [email]);
159
160     if (!user) {
161       res.status(404).send("Usuario no encontrado");
162       return;
163     }
164   }
165 }
```

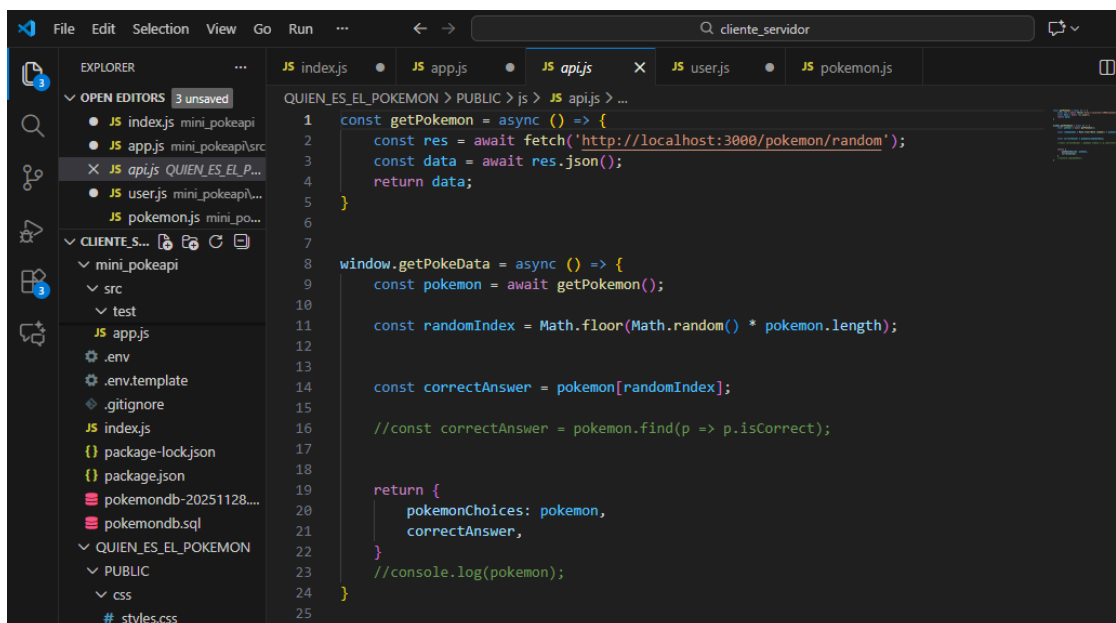


The screenshot shows the VS Code editor with the Explorer sidebar on the left. The Explorer sidebar shows the project structure: mini_pokeapi > src > controllers > JS userjs > loginUser. The main editor displays the code for the 'loginUser' function. The code is as follows:

```
196 const loginUser = async (req = request, res = response) => {
197   conn = await pool.getConnection();
198   const [user] = await conn.query(userQueries.users.viewByEmail, [email]);
199
200   if (!user) {
201     res.status(404).send("Usuario no encontrado");
202     return;
203   }
204
205   const validPassword = await bcrypt.compare(password, user.password);
206   if (!validPassword) {
207     res.status(401).send("Usuario o contraseña incorrectos");
208     return;
209   }
210
211   delete user.password; //Eliminar la contraseña antes de enviar la respuesta
212   res.status(200).send(user);
213 } catch (err) {
214   res.status(500).send("error");
215 } finally {
216   if (conn) conn.end();
217 }
218
219 module.exports = {
220   showUsers,
221   viewUser,
222   createUser,
223   removeUser,
224   updateUser,
225   loginUser
226 }
```

Este archivo es un controlador de Node.js robusto que gestiona todas las operaciones lógicas relacionadas con los usuarios. A diferencia de un simple repositorio de consultas, este programa implementa medidas de seguridad críticas, como la protección de contraseñas y validaciones de datos antes de interactuar con la base de datos.

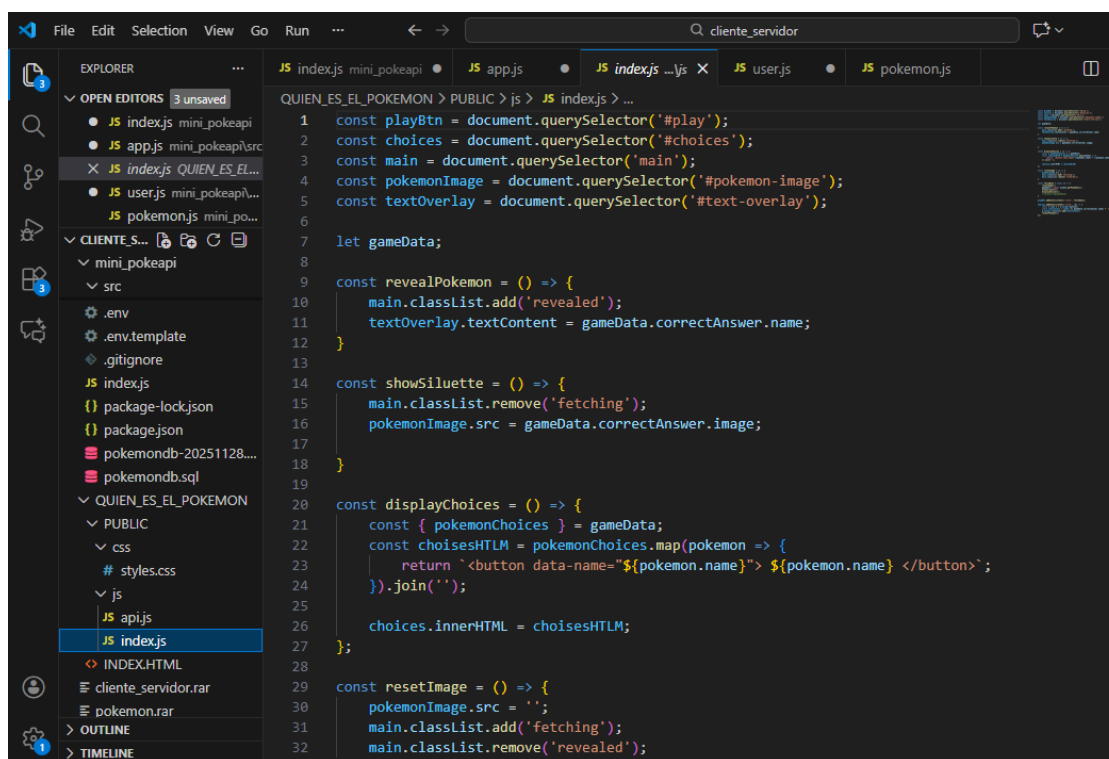
9. Comunicación



```
1 const getPokemon = async () => {
2   const res = await fetch('http://localhost:3000/pokemon/random');
3   const data = await res.json();
4   return data;
5 }
6
7
8 window.getPokeData = async () => {
9   const pokemon = await getPokemon();
10
11   const randomIndex = Math.floor(Math.random() * pokemon.length);
12
13   const correctAnswer = pokemon[randomIndex];
14
15   //const correctAnswer = pokemon.find(p => p.isCorrect);
16
17   return {
18     pokemonChoices: pokemon,
19     correctAnswer,
20   }
21 }
22
23 //console.log(pokemon);
24
25
```

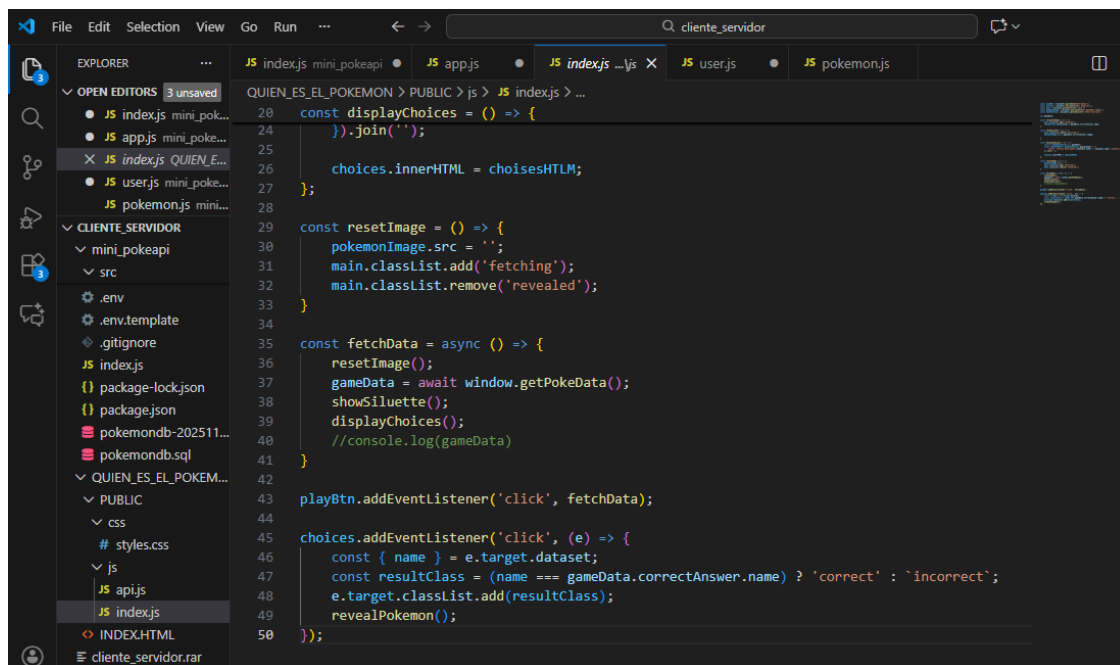
Este archivo, denominado api.js, actúa como la capa de comunicación del lado del cliente (Frontend). Su función principal es realizar peticiones al servidor local para obtener los datos necesarios y preparar la lógica de una trivia o juego de adivinanza de Pokémon.

10. Interfaz de Usuario



```
1 const playBtn = document.querySelector('#play');
2 const choices = document.querySelector('#choices');
3 const main = document.querySelector('#main');
4 const pokemonImage = document.querySelector('#pokemon-image');
5 const textOverlay = document.querySelector('#text-overlay');
6
7 let gameData;
8
9 const revealPokemon = () => {
10   main.classList.add('revealed');
11   textOverlay.textContent = gameData.correctAnswer.name;
12 }
13
14 const showSilhouette = () => {
15   main.classList.remove('fetching');
16   pokemonImage.src = gameData.correctAnswer.image;
17 }
18
19 const displayChoices = () => {
20   const { pokemonChoices } = gameData;
21   const choicesHTML = pokemonChoices.map(pokemon => {
22     return `<button data-name="${pokemon.name}"> ${pokemon.name} </button>`;
23   }).join('');
24   choices.innerHTML = choicesHTML;
25 }
26
27 const resetImage = () => {
28   pokemonImage.src = '';
29   main.classList.add('fetching');
30   main.classList.remove('revealed');
31 }
32
```

11.



```
File Edit Selection View Go Run ... cliente_servidor
EXPLORER
OPEN EDITORS 3 unsaved
  JS index.js mini_poke...
  JS app.js mini_poke...
  JS index.js QUIEN_E...
  JS user.js mini_poke...
  JS pokemon.js mini...
CLIENTE_SERVIDOR
  mini_pokeapi
  src
    .env
    .env.template
    .gitignore
    JS index.js
    {} package-lock.json
    {} package.json
    {} pokemondb-202511...
    {} pokemondb.sql
  QUIEN_ES_EL_POKEM...
    PUBLIC
      css
      # styles.css
      js
        JS api.js
        JS index.js
      INDEX.HTML
      cliente_servidor.rar

QUIEN_ES_EL_POKEMON > PUBLIC > js > JS index.js > ...
20 const displayChoices = () => {
21   choices.innerHTML = choicesHTML;
22   choices.innerHTML = choicesHTML;
23 };
24
25 const resetImage = () => {
26   pokemonImage.src = '';
27   main.classList.add('fetching');
28   main.classList.remove('revealed');
29 };
30
31 const fetchData = async () => {
32   resetImage();
33   gameData = await window.getPokeData();
34   showSilhouette();
35   displayChoices();
36   //console.log(gameData)
37 };
38
39 playBtn.addEventListener('click', fetchData);
40
41 choices.addEventListener('click', (e) => {
42   const { name } = e.target.dataset;
43   const resultClass = (name === gameData.correctAnswer.name) ? 'correct' : 'incorrect';
44   e.target.classList.add(resultClass);
45   revealPokemon();
46 });
```

Este archivo, denominado index.js, es el motor de la interfaz de usuario (Frontend). Se encarga de manipular el DOM (Document Object Model) para dar vida al juego de "Adivina el Pokémon", gestionando visualmente el progreso de la partida y las interacciones del jugador.

Al probar el programa tuve un problema con la base de datos, pues no pude conectarla con la api pues no fue posible importar la base de datos pero el programa esta correctamente pues no marca ningún error de sintaxis.